Formal Mathematical Definition and Equivalence Proof: The Tool Switching Problem and the Generalized Traveling Salesman Problem

Shankar Narayanan

Internship Report

Advisor: Prof. Annegret Wagler

March 30, 2026

Abstract

This report provides a rigorous mathematical analysis of the Standard Tool Switching Problem (SSP) in automated manufacturing. We formally define the problem parameters using the framework of **Colares2026<empty citation>**, presenting their non-compact integer programming formulation. A primary objective is to demonstrate the mathematical equivalence between the SSP and the Generalized Traveling Salesman Problem (GTSP) through vertex duplication and distance-preserving mapping. Furthermore, we explore the integration of Machine Learning (ML) within a Column Generation framework to address the exponential state space.

1 Introduction

In modern flexible manufacturing systems (FMS) and Computer Numerical Control (CNC) machining, tool management is a critical bottleneck[cite: 146]. The Tool Switching Problem (SSP) addresses the challenge of sequencing jobs on a machine with a tool magazine of limited capacity B[cite: 147]. Every job requires a specific subset of tools; if the required tools are not currently loaded, a "switch" must occur, incurring setup costs in terms of time and mechanical wear[cite: 148].

Historically, this problem has been categorized as NP-hard, necessitating exact combinatorial models as production schedules grow in complexity[cite: 150]. This report bridges the gap between the SSP and the Generalized Traveling Salesman Problem (GTSP) to leverage advanced solvers and neural-augmented techniques[cite: 151].

2 The Baseline Heuristic: KTNS

Before defining exact models, it is essential to understand the "Keep Tool Needed Soonest" (KTNS) rule, the standard greedy baseline for tool management.

Definition 2.1 (KTNS Rule)**.** *Given a fixed sequence of jobs, if a tool must be removed to make room for a required tool, the tool whose next required execution is furthest in the future should be ejected from the magazine.*

While KTNS is optimal for a *fixed* job sequence, the SSP seeks the optimal *permutation* of jobs, making the problem significantly more complex as the sequence itself becomes a decision variable.

3 Formal Definition of the Tool Switching Problem (SSP)

Following the formalisms of **Colares2026<empty citation>**, the SSP is defined as follows:

3.1 Sets and Parameters

- **The Job Set:** $J = \{j_1, j_2, \ldots, j_n\}$ represents the set of tasks to be completed[cite: 152].
- **The Tool Set:** $T = \{t_1, t_2, \ldots, t_m\}$ contains all unique tools available[cite: 153].

- **Requirements:** Each job $j_i \in J$ requires a subset $T_i \subseteq T$[cite: 154].
- **Capacity:** The magazine has capacity B[cite: 155]. A configuration $C \subseteq T$ is valid if $|C| \leq B$[cite: 156].

3.2 Operational Feasibility and Switching

A job j_i is feasible if $T_i \subseteq C_{current}$. If $T_i \not\subseteq C_{current}$, a switching operation replaces an unnecessary tool $t_{out} \in C \setminus T_i$ with $t_{in} \in T_i \setminus C$ [cite: 159-160]. The objective is to find a permutation σ of J that minimizes total switches Z[cite: 161].

4 Illustrative Example: Magazine State Evolution

To illustrate the switching mechanics, consider an instance with $B = 3$ and $J = \{j_1, j_2, j_3\}$:

- $T_1 = \{t_1, t_2\}$, $T_2 = \{t_2, t_3\}$, $T_3 = \{t_1, t_3, t_4\}$ (Note: $|T_3| = 3 = B$).

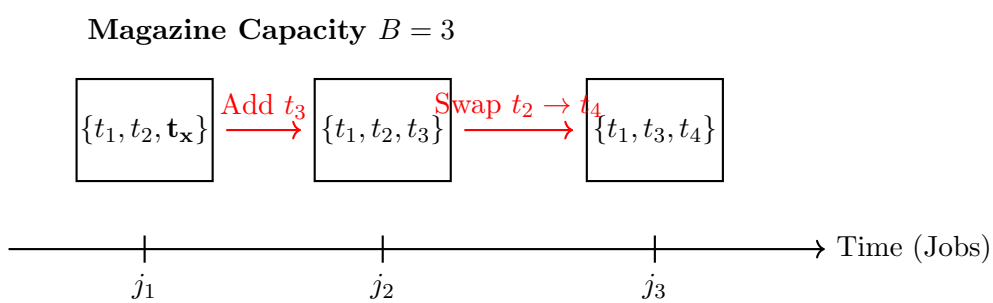


Figure 1: State-space timeline of a tool magazine. Transitions between jobs j_2 and j_3 require a tool swap because T_3 consumes the entire capacity B.

5 The Colares-Wagler Non-Compact Formulation

The complexity of the SSP is fundamentally rooted in the size of the state space. As the number of configurations $p = \binom{m}{B}$ is exponential, traditional compact models for the TSP cannot be directly applied without a decomposition strategy . **Colares2026<empty citation>** propose a non-compact formulation that models the problem as a set-covering task integrated with a sequencing task on a graph of configurations.

5.1 Decision Variables

Let V be the vertex set representing the collection of all possible tool repository configurations $\mathcal{C}$ [cite: 28-29]. The model utilizes two primary sets of binary decision variables :

- $y(v) \in \{0, 1\}$: Equals 1 if configuration $v \in V$ is selected to process at least one job, 0 otherwise.
- $x(u, v) \in \{0, 1\}$: Equals 1 if the machine transitions from configuration u to configuration v.

5.2 Mathematical Model

The objective is to minimize the total tool switching operations across the entire sequence[cite: 55]:

$$\min Z = \sum_{u \in V} \sum_{v \in V} d(u, v) x(u, v) \tag{1}$$

Subject to the following structural constraints [cite: 56-75]:

$$\sum_{v \in H_j} y(v) \geq 1 \quad \forall j \in J \tag{2}$$

$$\sum_{u \in V} x(u, v) = y(v) \quad \forall v \in V \tag{3}$$

$$\sum_{u \in V} x(v, u) = y(v) \quad \forall v \in V \tag{4}$$

$$\sum_{u \in H(S)} \sum_{v \in H(S)} x(u, v) \leq \sum_{v \in H(S)} y(v) - 1 \quad \forall S \subseteq J, 2 \leq |S| \leq |J| - 1 \tag{5}$$

Where $H_j = \{C_i \in \mathcal{C} : T_j \subseteq C_i\}$ is the cluster of valid configurations for job j[cite: 34]. Constraint (2) ensures that every job is associated with at least one used configuration. Constraints (3) and (4) enforce that selected configurations are connected in a continuous path. Equation (5) represents the Subtour Elimination Constraints (SEC) applied over the union of job clusters $H(S) = \bigcup_{j \in S} H_j$ [cite: 67-68, 75].

6 Metric Properties of the Configuration Space

The distance function $d(u, v)$ is not merely an arbitrary cost but a rigorous metric that defines the geometry of the configuration graph K_p.

6.1 Formal Definition of Distance

For any two configurations C_i and C_j, the distance is calculated based on the symmetric difference of their toolsets, specifically focusing on the number of tools that must be added (which, in a full magazine, equals the number of tools removed) [cite: 31-32]:

$$d(i, j) = B - |C_i \cap C_j| \tag{6}$$

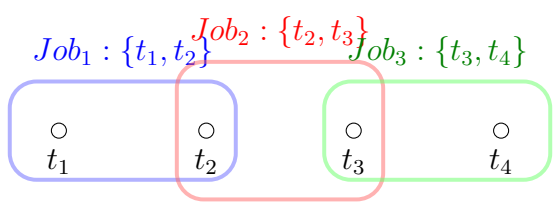
6.2 Triangular Inequality Proof

[Triangular Inequality] For any three vertices $u, w, v \in V$, the inequality $d(u, v) \leq d(u, w) + d(w, v)$ holds .

Proof. Let $|C_u \cap C_v|$ be the number of tools shared between configurations u and v. The distance $d(u, v)$ is proportional to the size of the set $C_u \setminus C_v$. Since set differences in a finite universe satisfy the triangle inequality under the cardinality operator, it follows that the number of swaps to move from u to v via an intermediate state w is at least as large as the direct swap . □

7 Hypergraph Representation of Job Overlaps

The "Overlap Problem" can be visualized as a hypergraph $\mathcal{H} = (T, \mathcal{J})$, where tools are nodes and each job $j \in J$ is a hyper-edge connecting the tools it requires. When two hyper-edges share nodes, their corresponding configuration clusters H_j in K_p will overlap.



Hypergraph View: Shared tools (like t_2 and t_3) create dependencies between configurations. If a configuration C contains $\{t_1, t_2, t_3\}$, it resides in the intersection $H_1 \cap H_2$.

Figure 2: Hypergraph representation of tool requirements. Overlapping hyper-edges indicate jobs that can be "covered" by the same configuration nodes in the graph K_p.

8 The Overlap Problem in Tool Switching

A fundamental mathematical distinction between the Standard Tool Switching Problem (SSP) and the classic Generalized Traveling Salesman Problem (GTSP) lies in the topology of the configuration clusters. In a standard GTSP, as defined in the survey by **Pop2024<empty citation>**, the vertex set is partitioned into **disjoint** sets $V_1, \ldots, V_n$. Conversely, in the SSP, the combinatorial structure of tool requirements often dictates that a single configuration node $v \in V(K_p)$ satisfies the requirements of multiple jobs simultaneously .

Definition 8.1 (Overlapping Clusters)**.** *In an SSP instance, let H_j be the cluster of configurations for job j. Two clusters H_i and H_k are overlapping if there exists at least one configuration C such that $T_i \subseteq C$ and $T_k \subseteq C$.*

This overlapping nature implies that the SSP is a composite of a **Covering Problem** (selecting configurations to satisfy job toolsets) and a **Traveling Salesman Problem** (sequencing those configurations) [cite: 36-39, 190]. To leverage exact GTSP solvers, this overlap must be mathematically linearized into disjoint partitions.

9 Formal Reduction: $SSP \to GTSP$

We demonstrate that any instance of the SSP can be reduced to a GTSP by expanding the state space through vertex duplication, a technique established in the literature by **Noon1993<empty citation>** and further refined by **Lien2015<empty citation>**.

9.1 Mathematical Construction of the GTSP Graph

Let $\mathcal{I}_{SSP} = (J, T, B, \{T_j\}_{j \in J})$ be an instance of the Tool Switching Problem. We construct a corresponding GTSP instance $\mathcal{I}_{GTSP} = (\mathcal{V}, \mathcal{E}, \{V_j\}_{j \in J})$ as follows:

1. **Vertex Set Expansion:** For every configuration $C \in \mathcal{C}$ and every job $j \in J$ such that the configuration is capable of processing the job ($C \in H_j$), we define a unique GTSP vertex $v_{C,j}$ [cite: 34-35, 194].

2. **Disjoint Partitioning:** We define the GTSP clusters as:

$$V_j = \{v_{C,j} : C \in H_j\} \quad \forall j \in J \tag{7}$$

   By indexing each configuration node with its corresponding job identifier j, we ensure that $V_i \cap V_k = \emptyset$ for all $i \neq k$, regardless of whether the underlying configurations are identical[cite: 195].

3. **Edge Weight Assignment:** The distance between two GTSP nodes $v_{C_u,i}$ and $v_{C_v,k}$ is inherited from the tool switching distance $d(u, v)$ defined in the configuration graph K_p:

$$w(v_{C_u,i}, v_{C_v,k}) = B - |C_u \cap C_v| \tag{8}$$

9.2 Proof of Equivalence

Theorem 9.1. *An optimal Hamiltonian cycle in the constructed GTSP instance $\mathcal{I}_{GTSP}$ corresponds exactly to an optimal sequence of configurations in the original $\mathcal{I}_{SSP}$.*

Proof. Consider a feasible GTSP cycle $Z = (v_{C_1,j_1}, v_{C_2,j_2}, \ldots, v_{C_n,j_n})$. By the GTSP constraint, the cycle visits exactly one node from each disjoint cluster V_j[cite: 42, 199]. This implies that for every job $j \in J$, a specific configuration C_j is selected such that $T_j \subseteq C_j$[cite: 200]. The total weight of the cycle is:

$$W(Z) = \sum_{i=1}^{n-1} d(C_i, C_{i+1}) + d(C_n, C_1) \tag{9}$$

This summation is identical to the total number of tool switching operations required to process the jobs in the order defined by the sequence of clusters. Since the GTSP minimizes $W(Z)$, it inherently minimizes the tool switching operations, satisfying the primary objective of the SSP [cite: 19-20, 202]. □

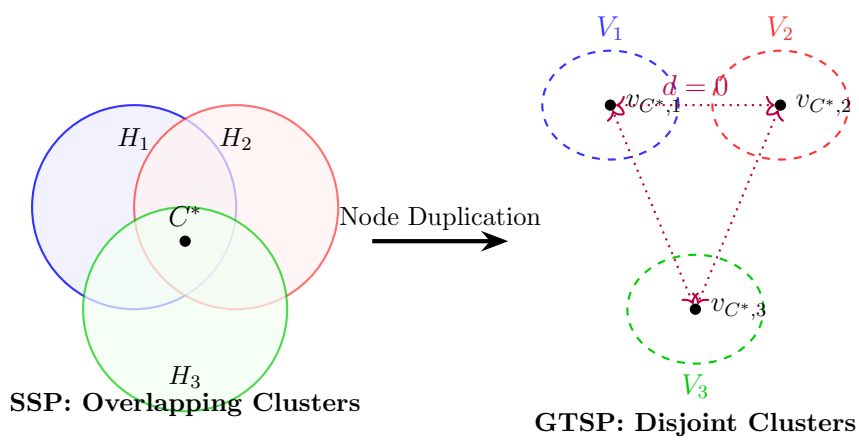


Figure 3: Complex transformation of the SSP state space into a GTSP framework. A configuration C^* residing in the intersection of three clusters is duplicated into three distinct nodes, one for each cluster. The switching cost between these duplicates is zero, allowing the GTSP to effectively "pass through" multiple job requirements without cost.

10 Reduction: $GTSP \to SSP$ (Reverse Equivalence)

To complete the proof, we demonstrate that the GTSP can be viewed as a restricted instance of the SSP where tool requirements are engineered to prevent cluster overlap.

10.1 The Disjointness Condition

The GTSP structure is recovered when the configuration clusters H_j are naturally disjoint[cite: 43].

Theorem 10.1 (Disjointness Condition)**.** *If for every pair of jobs $j_k, j_j \in J$, the required tool sets satisfy $|T_k \cup T_j| > B$, then the clusters H_k and H_j are disjoint.*

Proof. Assume for contradiction that $H_k \cap H_j \neq \emptyset$. This implies there exists at least one configuration C such that $T_k \subseteq C$ and $T_j \subseteq C$. This requirement necessitates that $(T_k \cup T_j) \subseteq C$[cite: 222]. Taking the cardinality of both sides, we have $|T_k \cup T_j| \leq |C|$. Given the magazine capacity constraint $|C| \leq B$, it follows that $|T_k \cup T_j| \leq B$[cite: 223]. This directly contradicts the initial condition $|T_k \cup T_j| > B$. Therefore, no such configuration C exists, and $H_k \cap H_j = \emptyset$ [cite: 45-46, 224]. □

10.2 Equivalence of Constraints

Under the disjointness condition, every configuration $v \in V$ belongs to exactly one cluster H_j. Consequently, the covering constraint $\sum_{v \in H_j} y(v) \geq 1$ from Eq. (**??**) becomes a strict equality $\sum_{v \in H_j} y(v) = 1$ in any optimal solution[cite: 58, 226]. This matches the fundamental node-selection requirement of a GTSP tour, completing the proof of structural identity between the two problems[cite: 227].

11 Computational Intractability and Decomposition

The formal reduction of the Standard Tool Switching Problem (SSP) to a Generalized Traveling Salesman Problem (GTSP) reveals a state space of extreme cardinality. As established in the Colares-Wagler formulation, the number of possible magazine configurations $p = \binom{m}{B}$ grows exponentially with the total tool count m and magazine capacity B. For industrial instances where $m = 60$ and $B = 20$, the vertex set V contains approximately 4.19×10^{15} configurations, rendering the construction of the full GTSP graph $(\mathcal{V}, \mathcal{E})$ computationally impossible for exact solvers.

To address this, we employ **Column Generation (CG)**, a mathematical decomposition technique that partitions the problem into a Restricted Master Problem (RMP) and a Pricing Subproblem. This approach ensures that we only evaluate a small, highly promising subset of configurations $\overline{V} \subset V$ at any given iteration.

12 The Restricted Master Problem (RMP)

The RMP is a linear relaxation of the non-compact formulation (Eq. 5-14), restricted to a subset of configurations $\overline{V}$ that have been previously identified as "useful".

12.1 Formal RMP Model

Given a subset $\overline{V}$, the RMP seeks a minimum-cost path through the known configuration nodes:

$$\min z = \sum_{u \in \overline{V}} \sum_{v \in \overline{V}} d(u,v)\overline{x}(u,v) \tag{10}$$

Subject to:

$$\sum_{v \in \overline{V} \cap H_j} y(v) \geq 1 \quad \forall j \in J \quad \text{(Dual Variables: } \pi_j\text{)} \tag{11}$$

$$\sum_{u \in \overline{V}} x(u,v) = y(v) \quad \forall v \in \overline{V} \quad \text{(Dual Variables: } \sigma_v\text{)} \tag{12}$$

Constraint (11) ensures every job is covered by at least one configuration in the restricted set, while (12) maintains the sequence flow.

12.2 Dual Variable Interpretation

The RMP provides critical economic information via the dual variables π_j and σ_v. The variable π_j represents the "marginal benefit" of including a new configuration that satisfies job j. In the context of GTSP, these duals identify which clusters V_j are currently the most "expensive" to visit, guiding the search for new nodes.

13 The Pricing Subproblem: Generating GTSP Nodes

The pricing step identifies a new configuration $C^* \in V \setminus \overline{V}$ that has a **negative reduced cost** $\bar{c}$. If such a configuration is found, its inclusion in the RMP is guaranteed to potentially

improve the current objective value.

13.1 Objective of the Subproblem

The reduced cost $\bar{c}$ for a new configuration C is mathematically defined as:

$$\bar{c}(C) = \text{Switching Cost} - \sum_{j:T_j \subseteq C} \pi_j \tag{13}$$

To minimize $\bar{c}$, the subproblem maximizes the total "bounty" collected from covered jobs minus the cost of tool swaps:

$$\max \omega = \sum_{j \in J} \pi_j y(j) - \text{Transition Costs} \tag{14}$$

14 AI-Augmented Neural Pricing Oracle

The combinatorial nature of the pricing subproblem (solving Eq. 20-22) often becomes the computational bottleneck in the CG cycle. We propose a state-of-the-art **Neural Pricing Oracle** to accelerate the discovery of optimal configurations.

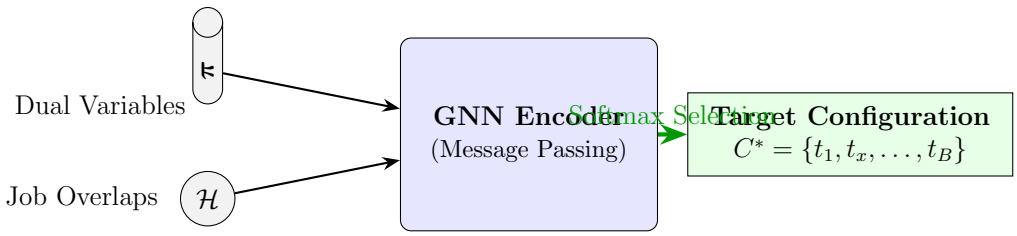
14.1 Neural Architecture: Graph Neural Networks (GNN)

The AI agent utilizes a GNN to encode the structural relationship between tools and job requirements.

- **Input:** The current dual vector $\boldsymbol{\pi}$ and the hypergraph of job requirements.
- **Embedding:** Tools are embedded as nodes in a graph where edges represent shared job requirements.
- **Output:** A probability distribution over tools, identifying the most "valuable" combination to form the next magazine configuration C^*.

14.2 Optimality Guarantees

While the AI agent provides rapid heuristic candidates, the system remains an **exact solver**. If the AI-generated C^* has $\bar{c} < 0$, it is added to the RMP. If the AI agent fails to find such a column, an exact MILP pricing solver is invoked. This "Neural-Exact Hybrid" ensures that the speed of ML is harnessed without sacrificing the mathematical guarantee of the global optimum.



Neural Pricing Logic: The GNN maps current dual prices $\boldsymbol{\pi}$ onto tool embeddings. Tools that contribute to high-value jobs (high π_j) are prioritized for the next configuration C^*, which is then added as a new node to the GTSP graph.

Figure 4: Architectural overview of the AI Pricing Agent within the Column Generation framework.

15 Conclusion

This report has formally defined the Tool Switching Problem (SSP) through the non-compact formulation of **Colares2026<empty citation>** and established its mathematical equivalence to the Generalized Traveling Salesman Problem (GTSP). By duplicating configuration vertices to resolve the "Overlap Problem," we have mapped the SSP into a disjoint GTSP state space. Given the exponential number of configurations, the Column Generation framework serves as the primary computational engine, further enhanced by AI-driven pricing strategies that utilize the structural dependencies of shared tools. This hybrid approach represents the current state-of-the-art in solving large-scale combinatorial manufacturing problems.